

QUESTION 8(a)

Question:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 500

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            /* Handle ${VARIABLE} */
            if (input[i] == '{')
            {
                i++;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;
            }
        }
        output[j++] = input[i++];
    }
    output[j] = '\0';
}
```

```

    }
    /* Handle $VARIABLE */
    else
    {
        while (input[i] != '\0' &&
              (isalnum(input[i]) || input[i] == '_') &&
              k < 99)
        {
            variable[k++] = input[i++];
        }

        variable[k] = '\0';
    }

    if (k > 0)
    {
        char *value = getenv(variable);

        if (value != NULL)
        {
            for (int x = 0;
                 value[x] != '\0' && j < MAX - 1;
                 x++)
            {
                output[j++] = value[x];
            }
        }
        else
        {
            printf("Undefined variable: %s\n", variable);
        }
    }
    else
    {
        output[j++] = '$';
    }
}

```

```

    }
    else
    {
        output[j++] = input[i++];
    }
}

output[j] = '\0';
}

int main()
{
    char input[MAX];
    char output[MAX];

    printf("==== Variable Expansion =====\n");

    printf("Enter text: ");
    fgets(input, MAX, stdin);

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("\nOriginal Token : %s\n", input);
    printf("Expanded Token : %s\n", output);

    return 0;
}

```

Output

```

Executable: /usr/bin/ls
sri_dhruthi@Dhruthi:~$ nano skill8a.c
sri_dhruthi@Dhruthi:~$ gcc skill8a.c -o skill8a
sri_dhruthi@Dhruthi:~$ ./skill8a
==== Variable Expansion =====
Enter text: Hello $USER

Original Token : Hello $USER
Expanded Token : Hello sri_dhruthi
sri_dhruthi@Dhruthi:~$ |

```

QUESTION 8(b)

Question:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX 100

void cmd_pwd()
{
    char path[MAX];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current Directory: %s\n", path);
    else
        perror("pwd");
}

void cmd_cd(char *path)
{
    if (path == NULL)
    {
        printf("Usage: cd <directory>\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
}

void cmd_echo(char *text)
{
    if (text != NULL)
        printf("%s\n", text);
}

void cmd_help()
{

```

```
int main()
{
    char input[MAX];
    char *command;
    char *argument;

    /* Dispatch table */
    struct Builtin
    {
        char *name;
        void (*function)();
    };

    struct Builtin builtins[] =
    {
        {"pwd", cmd_pwd},
        {"help", cmd_help},
        {NULL, NULL}
    };

    int builtin_count = 2;

    printf("==== Built-in Command Shell ==== \n");
    printf("Type 'help' to see commands.\n");

    while (1)
    {
        printf("\n$ ");

        if (fgets(input, MAX, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        command = strtok(input, " ");
        argument = strtok(NULL, "");
    }
}
```

```

printf("Type 'help' to see commands.\n");

while (1)
{
    printf("\n$ ");

    if (fgets(input, MAX, stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    command = strtok(input, " ");
    argument = strtok(NULL, "");

    if (command == NULL)
    {
        continue;
    }

    /* Exit */
    if (strcmp(command, "exit") == 0)
    {
        printf("Exiting shell...\n");
        break;
    }

    /* cd needs an argument */
    if (strcmp(command, "cd") == 0)
    {
        cmd_cd(argument);
        continue;
    }

    /* echo */
    if (strcmp(command, "echo") == 0)
    {
        cmd_echo(argument);

```

⌂ Help ⌂ Write Out ⌂ Where Is ⌂ Cut ⌂

```

/* Search dispatch table */
int found = 0;

for (int i = 0; i < builtin_count; i++)
{
    if (strcmp(command, builtins[i].name) == 0)
    {
        builtins[i].function();
        found = 1;
        break;
    }
}

if (!found)
{
    printf("Invalid command: %s\n", command);
}

return 0;
}

```

Output

```
sri_dhruthi@Dhruthi:~$ nano skill8b.c
sri_dhruthi@Dhruthi:~$ gcc skill8b.c -o skill8b
sri_dhruthi@Dhruthi:~$ ./skill8b
==== Built-in Command Shell ====
Type 'help' to see commands.

$ pwd
Current Directory: /home/sri_dhruthi

$ cd
Usage: cd <directory>

$ echo hello world
hello world

$ help

Built-in Commands:
cd <dir>    - Change directory
pwd         - Show current directory
echo <text> - Display text
help       - Show available commands
exit       - Exit program

$ exit
Exiting shell...
sri_dhruthi@Dhruthi:~$ |
```